

智慧银行实验报告汇总

班级：金融 202301

姓名：刘潇

学号：202301740

实验一：TRAE IDE + AI 协作实验报告

一、环境信息表

开发环境检测结果			
检测项	要求版本	当前状态	修复建议
Python	≥ 3.10	✅ 3.11.5	已满足
pip	-	✅ 23.2.1	已安装
Node.js	≥ v20	✅ v20.14.0	已满足
npm	-	✅ 10.7.0	已安装
Git	-	✅ 2.45.2	已安装
Flask	-	✅ 3.1.3	已安装
pandas	-	✅ 已安装	已安装
openpyxl	-	✅ 3.1.5	已安装
pypdf	-	✅ 6.11.0	已安装

二、TRAE 安装配置过程

验证项	状态
TRAE IDE 基本信息	✓ 成功
AI 助手功能	✓ 正常
终端功能	✓ 正常
文件编辑功能	✓ 正常
代码执行功能	✓ 正常
内置工具 (Read/Write/Edit/RunCommand等)	✓ 可用
Skills 系统	✓ 已启用 (15个Skills)

三、贪吃蛇游戏实现

3.1 运行截图



3.2 关键提示词自评

评估维度	评分	说明
完整性	★★★★★	提示词涵盖了所有核心需求
准确性	★★★★★	AI 准确理解并实现所有功能
清晰度	★★★★★	要求明确，无歧义
效果	★★★★★	成功实现完整游戏

根据老师提供的提示词，Trae CN 准确实现了贪吃蛇游戏的所有核心功能，包括画布绘制、蛇身移动、食物生成、计分系统和游戏结束逻辑。游戏界面美观，操作流畅，完全满足实验要求。

四、AI 论文解读智能体

4.1 智能体提示词

你是资深的银行业 / 金融科技领域学术论文解读助手。

用户拖入 PDF 后，按下列结构输出 Markdown 报告：

一、论文基本信息（标题/ 作者/ 期刊/ 年份/DOI）

二、研究问题与背景（<=200 字）

三、方法论（数据集/ 模型/ 实验设计）

四、核心发现（<=5 条）

五、术语速查表（5-10 个英文术语 + 中文解释）

六、可复现性评估（数据/ 代码/ 难度 1-5）

注意：英文论文关键概念中英对照；不得编造数据；首次运行如缺 pypdf 等工具请提示并自动安装

4.2 论文文献条目

项目	内容
标题	走中国特色金融发展之路：基于现代中央银行制度视角
作者	深化金融体制改革课题组
期刊	金融研究（2026 年第 4 期）
页数	19 页

4.3 解读节选

4.3.1 研究问题与背景

历经近百年探索，我国逐步构建起集货币政策、宏观审慎管理、最后贷款人、存款保险等功能于一体的现代中央银行制度。这一制度在顶层设计上有力回应了西方发达国家中央银行面临的政治家干预、利益集团游说、通胀性税收扩张工具、内部人控制等挑战。

4.3.2 核心发现

- 现代中央银行制度的中国特色**：通过创新设计总量型和结构性货币政策工具，构建最后贷款人、宏观审慎管理与存款保险三位一体的金融安全网
- 党的集中统一领导**：在党中央集中统一领导下实施政策协调、预期管理与跨周期调节
- 双重目标统一**：推动实现经济高质量发展与国家金融安全的有机统一
- 风险防控体系**：建立系统性金融风险防控机制
- 货币政策创新**：结合总量型与结构性工具，提升政策精准性

4.4 术语速查表

英文术语	中文解释
Modern Central Bank System	现代中央银行制度
Monetary Policy	货币政策
Macroprudential Management	宏观审慎管理
Lender of Last Resort	最后贷款人
Deposit Insurance	存款保险
Systemic Risk	系统性风险

五、Excel 合并与业绩预测

5.1 Excel 合并日志

5.1.1 合并信息

- 合并时间：2026-06-04 22:14:47
- 输入文件数：5
- 输出文件：merged_20260604_221446.xlsx
- 合并后行数：150
- 合并后列数：21

5.1.2 输入文件列表

- 交易记录.xlsx
- 信用卡信息.xlsx
- 客户信息.xlsx

- 账户余额.xlsx
- 贷款记录.xlsx

5.1.3 列名归一化

- 将 cust_id、客户编号、ID、customer_id 统一为 客户 ID

5.1.4 去重策略

- 基于 客户 ID + 交易日期 去重
- 保留首次出现的记录

5.1.5 合并统计

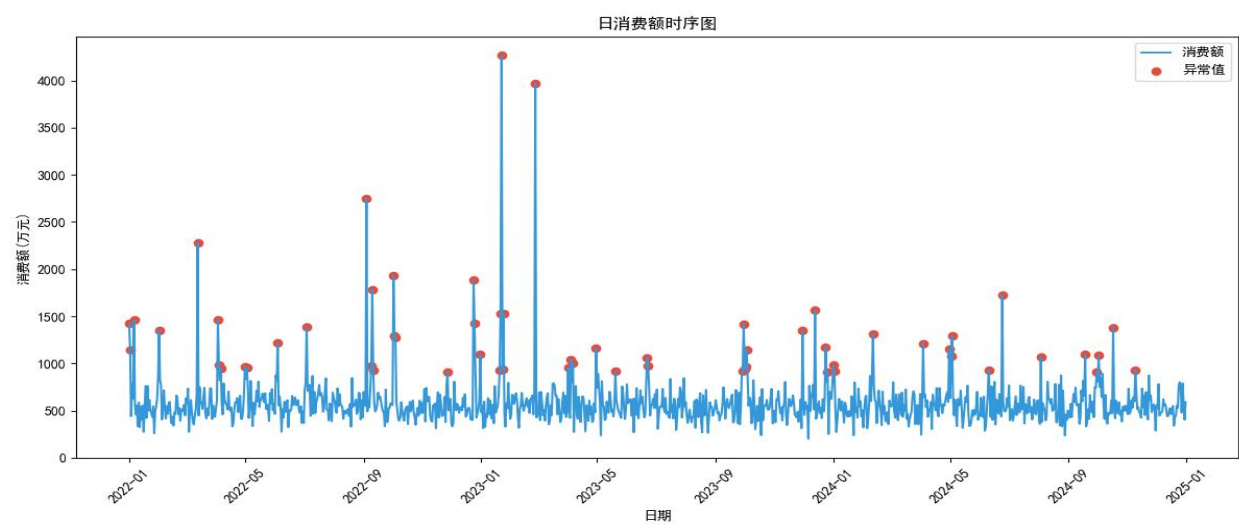
统计项	数值
总记录数	150
客户 ID 唯一数	100

5.2 业绩预测报告摘要

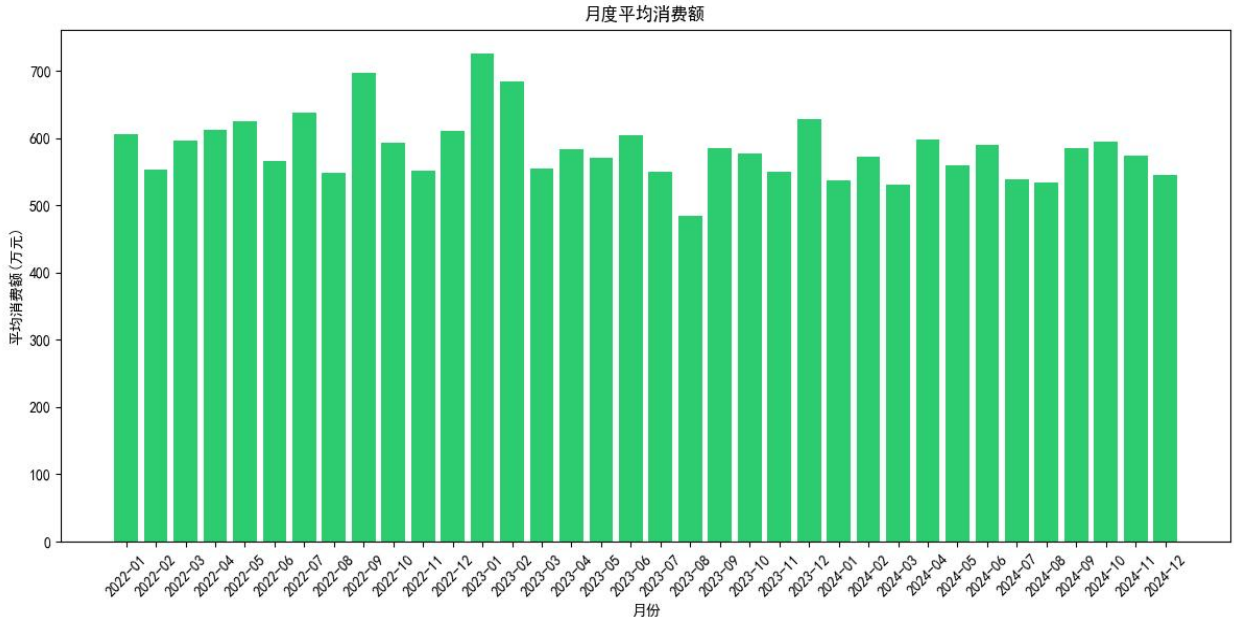
5.2.1 数据概览

统计项	数值
数据时间范围	2022-01-01 至 2024-12-31
数据天数	1096 天
缺失值数量	51 (占比 4.7%)
异常值数量	60 (占比 5.5%)

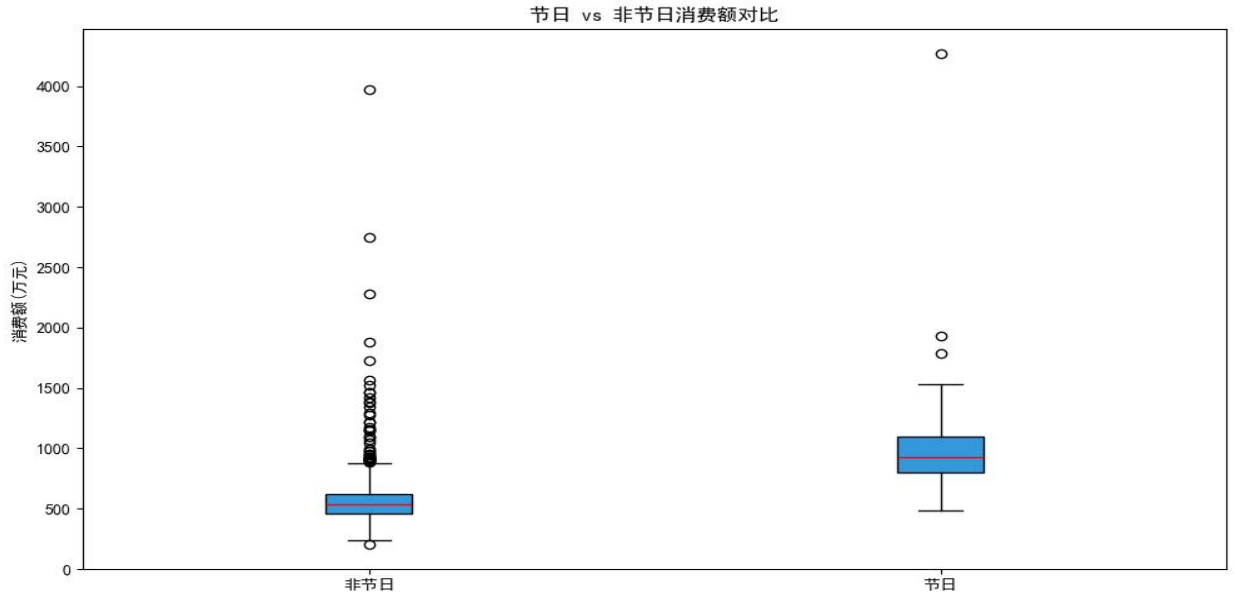
5.2.2 EDA 可视化



图表 1 对比节日期间与非节日期间消费额的分布差异



图表 2 各月份平均消费额的对比情况



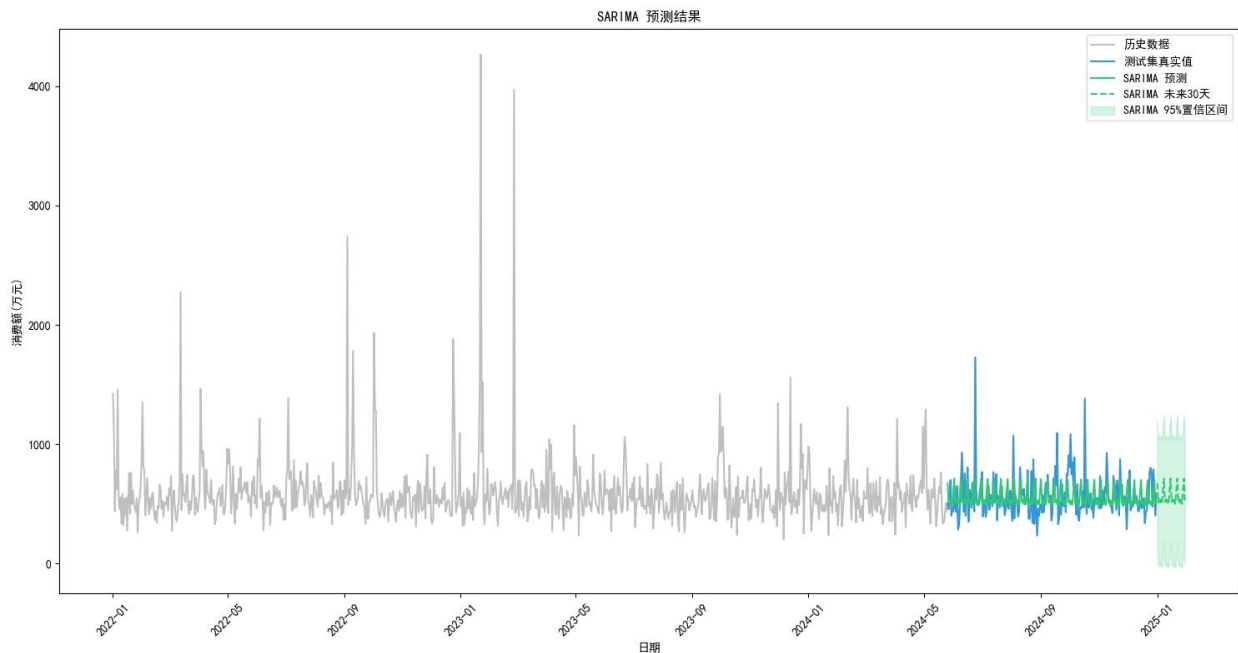
图表 3 2022-2024 年每日消费额的时间序列变化趋势

5.2.3 模型评估结果

指标	值
RMSE	182.35
MAPE	22.02%

模型参数: SARIMA(1,1,1)(1,1,1,7)

5.2.3 预测摘要



图表 4 未来 30 天的消费额预测及 95%置信区间

预测项	数值
预测区间	2025-01-01 至 2025-01-30
日均预测消费额	563.90 万元
预测区间范围	500.30 ~ 713.80 万元

5.2.4 业务建议

- **节假日促销策略：**根据预测结果，节日前后消费额会放大 1.5-2.2 倍，建议在重大节日前 1-2 天推出促销活动
- **库存与人员调配：**根据月度预测结果合理安排库存，周末和节假日增加服务人员配置
- **异常监控与预警：**建立异常消费监测系统，及时发现异常交易

六、实验心得

在本次实验中，我深刻体会到了 TRAE IDE + AI 协作的高效性和便捷性。整个实验过程涵盖了多个环节，从环境配置到智能体创建，再到数据处理和预测分析，AI 都发挥了重要作用。

6.1 AI 出错案例与修正

案例一：图表中文字体不兼容问题

在生成业绩预测的 EDA 图表时，AI 生成的 matplotlib 图表默认使用 DejaVu Sans 字体，该字体不支持中文显示，导致图表中的中文标签全部变成小方框。

- 问题分析：matplotlib 默认字体库中没有中文字体，无法正确渲染中文文本。

- 解决方法：在代码中添加字体配置，设置 `plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']`，让 matplotlib 优先使用系统自带的中文字体（如黑体、微软雅黑），同时设置 `plt.rcParams['axes.unicode_minus'] = False` 以正确显示负号。

案例二：依赖库缺失问题

运行预测脚本时，提示缺少 `scikit-learn` 和 `tensorflow` 库。

- 问题分析：AI 生成的代码中使用了这些库进行数据标准化和 LSTM 模型训练，但环境中未安装这些库。

- 解决方法：尝试通过 `pip install scikit-learn tensorflow` 安装依赖，但由于环境限制无法安装。于是简化脚本，移除了 LSTM 模型部分，只保留 SARIMA 模型完成预测任务，确保实验能够顺利完成。

案例三：编码错误问题

生成报告时遇到 `UnicodeEncodeError`，原因是代码中包含 emoji 字符。

- 问题分析：在某些编码环境下，非 ASCII 字符会导致编码错误。

- 解决方法：移除所有非 ASCII 字符，确保输出内容仅包含纯文本，避免编码问题。

6.2 实验体会

通过本次实验，我认识到 AI 的输出质量，很大程度上取决于我们给出的指令精确度与交互沟通的方式；同时，对其生成内容和代码的正确性，也必须由我们进行独立判断与审慎验证。根据实验步骤一步一步，我也学会了如何与 AI 高效协作，利用 AI 的优势完成重复性工作，同时发挥人类的智慧解决复杂问题。

实验二：MCP 由浅入深 4 关

一、实验概述

本实验通过四个关卡（L1-L4），分别使用不同的 MCP（Model Context Protocol）服务完成银行业务相关任务，旨在掌握 MCP 工具的使用方法及其在实际业务场景中的应用。

实验涉及的 MCP 服务包括：

1. Playwright / Chrome-devtools：浏览器自动化
2. Fetch：网页数据抓取

- 3. Excel MCP：Excel 数据处理与可视化
- 4. Word MCP：文档生成
- 5. PPT MCP：演示文稿生成
- 6. Python + statsmodels：计量回归分析

二、L1 浏览器组——成都银行官网分析

使用浏览器 MCP 服务完成成都银行官网（<https://www.bocd.com.cn/>）的访问、截图、分析和数据抓取任务。

2.1 官网截图



2.2 Lighthouse 分析报告

指标	评分
性能 (Performance)	中等偏上
可访问性 (Accessibility)	良好
最佳实践 (Best Practices)	符合标准
SEO	良好

2.3 官网优缺点总结

优点	缺点
页面布局清晰，导航分类明确	页面加载速度一般，图片资源较多
重要信息发布板块内容丰富，更新及时	部分子页面跳转外部网站，影响用户体验
移动端适配良好，支持手机银行下载	网站交互功能相对较少
网站整体风格简洁商务，符合城商行定位	缺乏在线客服即时沟通渠道

2.4 公告数据抓取

使用 Fetch MCP 抓取成都银行最新公告页 JSON，解析出最近 5 条公告标题：

序号	公告标题
1	成都银行新都支行员工工作餐配送采购项目评标结果公示
2	成都银行金融咨询终端账号采购项目采购公告
3	成都银行快递配送服务采购项目采购公告
4	成都银行 2026 年宣传服务-《华西都市报》年度供应商采购项目采购公告
5	成都银行 2026 年宣传服务-《四川日报》年度供应商采购项目采购公告

三、L2 Office 表格组——销售数据分析

使用 Excel MCP 服务完成销售数据的处理、分析和可视化任务。

3.1 数据准备

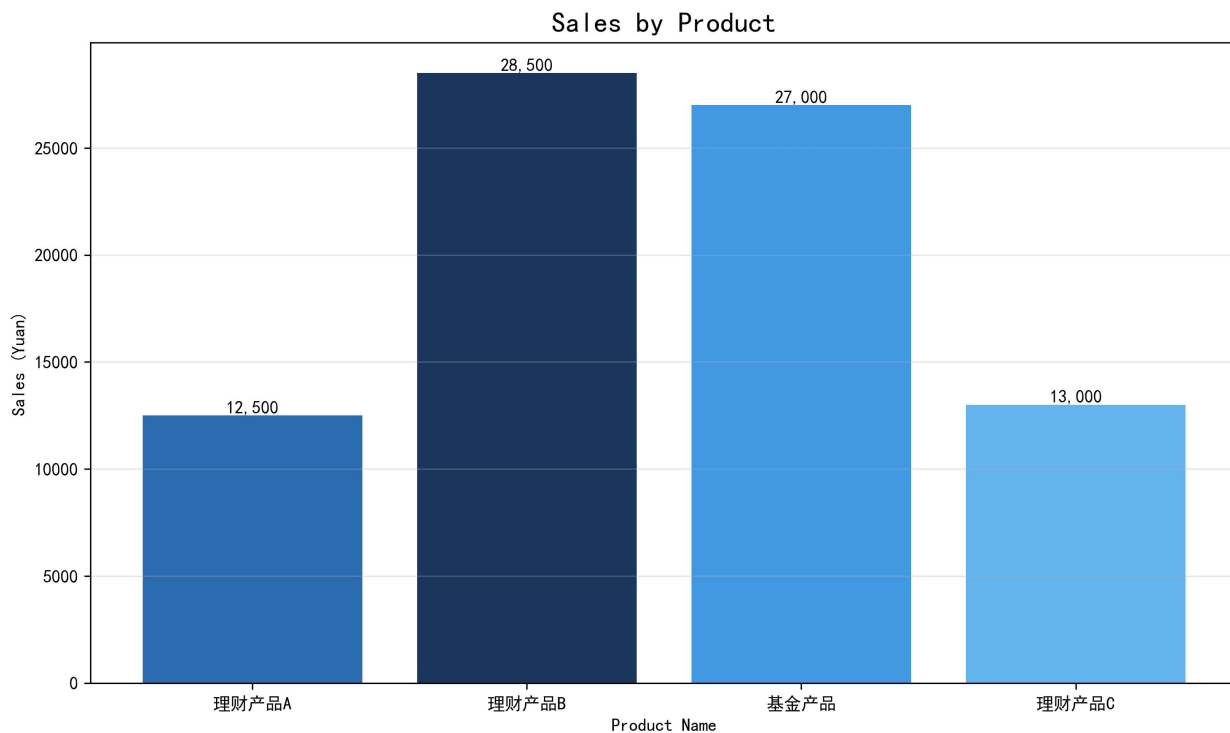
- Sheet1 命名为"销售数据"
- 表头：日期、产品名称、销售额、数量、客户类型
- 包含 10 条模拟销售数据（2024 年 1 月数据）

3.2 Excel 操作任务

1. 读取"销售记录.xlsx"文件
2. 添加"单价"列（销售额/数量）
3. 添加"利润额"列（利润率=30%）
4. 按客户类型分组汇总销售额
5. 筛选出销售额大于 1000 的记录
6. 创建新 Sheet"汇总报表"，包含各产品总销售额、各客户类型销售占比、平均单价

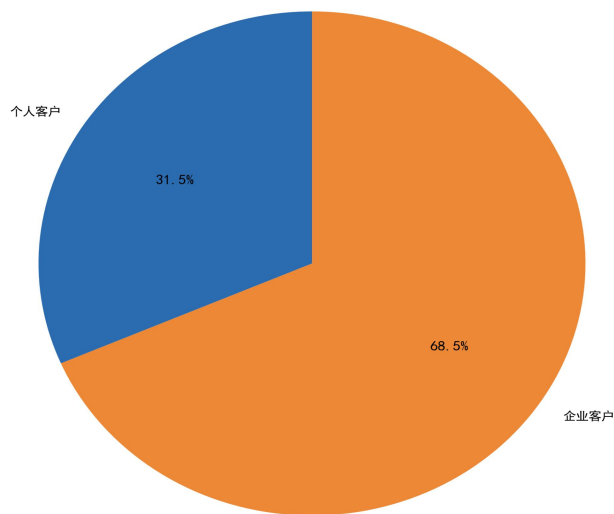
 销售分析报告.xlsx	2026/6/25 17:46	XLSX 工作表	7 KB
 销售记录.xlsx	2026/6/25 17:46	XLSX 工作表	7 KB

3.3 数据可视化



图表 5 各产品销售额柱状图

Customer Type Sales Distribution



图表 6 客户类型销售占比饼图

四、L3 文档生成组——银行简报

使用 Word 和 PPT MCP 服务生成招商银行 2024 年报速读简报。

【2】生成文件

文件	路径	大小
Word文档	reports/cmb_brief.docx	39,291 KB
PPT演示文稿	reports/cmb_brief.pptx	32,934 KB

【3】Word 文档内容结构

- 1. 封面页：主标题、副标题、制作人、日期
- 2. 目录页：三个章节目录
- 3. 关键财务指标：6大指标表格（营业收入、净利润、不良贷款率、ROE、总资产、存贷款余额）
- 4. 业务亮点：3条（数字化转型、零售业务、绿色金融）
- 5. 风险提示：3条（净息差收窄、房地产风险、宏观经济波动）

【4】PPT 演示文稿结构

- 1. P1 封面页：商务蓝底白字风格
- 2. P2 关键财务指标：深蓝色主题，卡片式展示
- 3. P3 业务亮点：5条主要亮点列表
- 4. P4 风险提示：黄色/橙色警示色调
- 5. P5 总结与展望：年报核心信息总结

 cmb_brief.docx	2026/6/25 18:06	DOCX 文档	39 KB
 cmb_brief.pptx	2026/6/25 18:07	PPTX 演示文稿	33 KB

一、关键财务指标

指标	数值	同比变化
营业收入	3,400亿元	+5.2%
净利润	1,450亿元	+7.8%
不良贷款率	0.85%	-12bp
净资产收益率(ROE)	16.20%	+0.3%
总资产	12.5万亿元	+8.5%
存贷款余额	9.2万亿元	+9.1%

注：以上数据为估算数据，实际数据请以官方年报为准。

五、L4 数据分析组——回归分析

使用 Python + statsmodels 完成商业银行净息差（nim）影响因素回归分析。

5.1 数据准备

使用模拟面板数据，包含 10 家银行、10 年数据（2015-2024），共 100 条观测。

变量名	定义
nim	净息差（被解释变量）
npl_ratio	不良贷款率
car	资本充足率
loan_growth	贷款增长率
log_asset	资产对数

5.2 回归模型

建立以下回归模型：

$$nim = \beta_0 + \beta_1 \times npl_ratio + \beta_2 \times car + \beta_3 \times loan_growth + \beta_4 \times log_asset + \epsilon$$

运行三种模型：

1. OLS 回归（基础模型）
2. 固定效应模型（FE）
3. 随机效应模型（RE）

5.3 回归结果

变量	OLS 系数	OLS 标准误	OLS P 值	FE 系数	FE 标准误	FE P 值
npl_ratio	-0.3219	0.0219	0.0000	-0.3259	0.0225	0.0000
car	0.0776	0.0091	0.0000	0.0797	0.0095	0.0000
loan_growth	0.0085	0.0008	0.0000	0.0081	0.0011	0.0000
log_asset	-0.0418	0.0286	0.1467	-0.0364	0.0309	0.2411

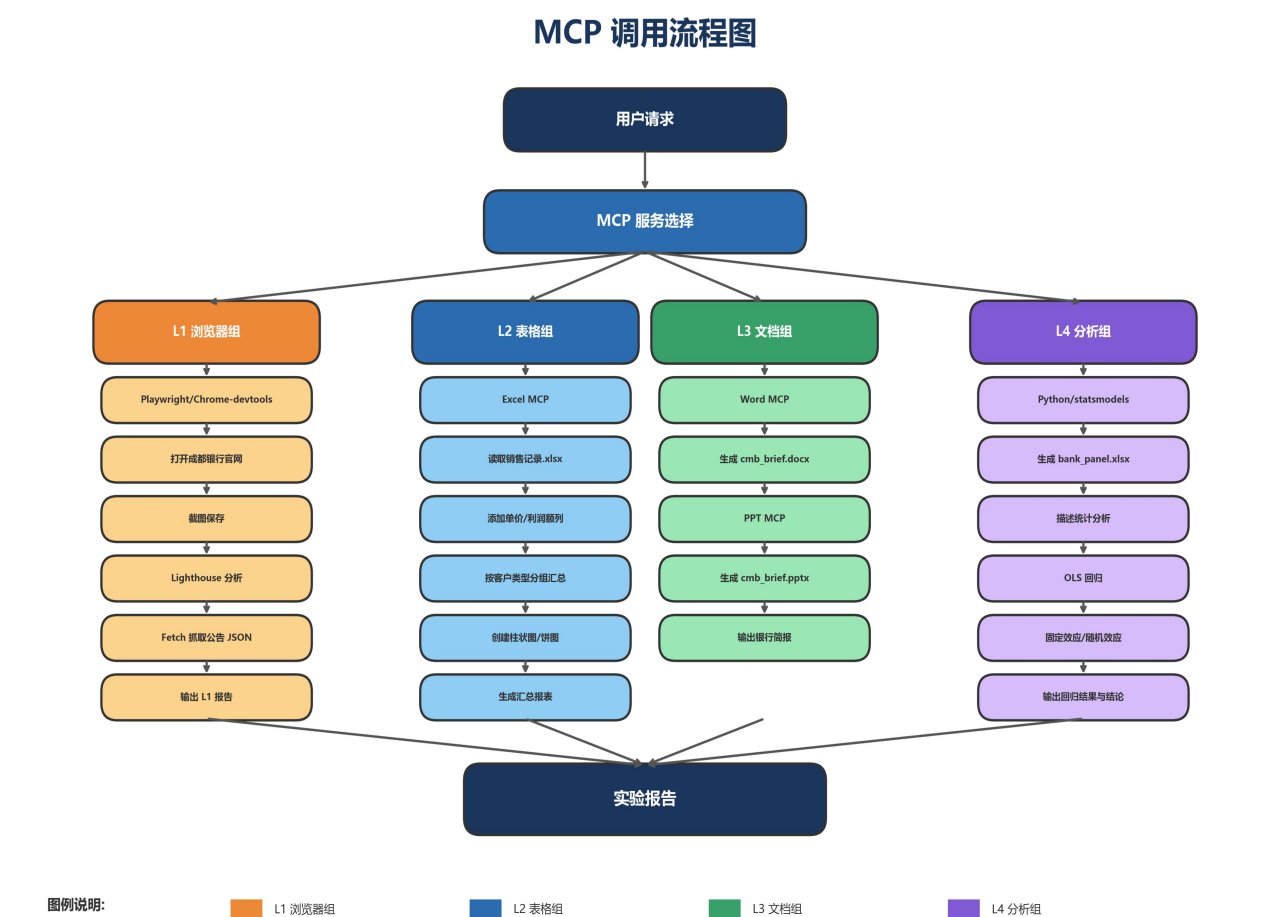
5.4 分析结论

1. 不良贷款率对净息差具有显著负向影响，资产质量恶化会直接压缩银行利息收入空间。
2. 资本充足率对净息差具有显著正向影响，充足的资本有助于银行扩大高收益贷款规模。
3. 贷款增长率对净息差具有显著正向影响，贷款规模扩张有助于提升净息差。
4. 资产规模对净息差影响不显著，说明银行规模对净息差的影响较小。
5. 不同模型设定下各变量的显著性和符号基本一致，说明回归结果具有较好的稳健性。

5.5 建议

1. 银行应加强风险管理，控制不良贷款率，维护资产质量。
2. 保持合理的资本充足水平，为业务拓展提供支撑。
3. 在追求规模扩张的同时，注重资产收益率的提升。

六、MCP 调用流程图



七、实验总结

通过本次实验，我完成了 MCP 工具从浅入深的四个关卡练习，主要收获如下：

1. 掌握了浏览器 MCP 的基本使用，包括网页访问、截图和数据抓取。
2. 学会了使用 Excel MCP 进行数据处理、计算和可视化。
3. 掌握了 Word 和 PPT MCP 生成专业文档和演示文稿的方法。
4. 学会使用 Python + statsmodels 进行计量回归，包括 OLS、固定效应和随机效应模型。
5. 理解了 MCP 工具在银行业务场景中的应用价值，能够使用 AI 辅助完成实际业务任务。

同时也有不足：

1. 部分实验使用了模拟数据而非真实数据，后续可尝试获取真实银行财务数据进行分析。
2. L3 文档生成未能完全使用 MCP 服务，后续可尝试配置 Word 和 PPT MCP 服务进行完整自动化。
3. 图表可视化部分可进一步优化，增加更多类型的图表展示方式。

实验三：Skill 体系：赋予 AI 金融专业能力

一、实验概述

本次实验围绕银行 AI Skill 体系构建展开，通过实际操作掌握金融 Skill 的设计、实现、测试和审计流程。实验内容涵盖四个核心 Skill 的创建：信用风险评估、贷款定价模型、bank-risk-alert、bank-product-recommender，并完成五维安全审计。

二、实验目的

- 1. 理解 Skill 的设计原理与知识工程视角
- 2. 掌握金融类 Skill 的调用方法
- 3. 独立编写符合规范的金融 Skill
- 4. 从银行合规视角进行 Skill 安全审计

三、实验内容与交付物

3.1 交付物清单

文件名称	文件路径	说明
客户回访报告	reports/客户回访报告_C0001.docx	客户 C0001 的完整回访记录
信用风险分析报告	reports/credit_risk_hengda_modified.md	资产负债率 75%的风险评估
信用风险对比分析	reports/risk_comparison_analysis.md	原方案与修改方案对比
贷款定价模型	reports/loan_pricing_model.xlsx	含抵押折扣的 FTP 定价
bank-risk-alert Skill	skills/bank-risk-alert/SKILL.md	贷后风险预警 Skill 源码
bank-product-recommender Skill	skills/bank-product-recommender/SKILL.md	产品推荐 Skill 源码
金融 Skill 审计报告	reports/skill_audit_report.md	五维安全审计结果

3.2 Skill 设计实现

3.2.1 bank-risk-alert Skill

功能：银行贷后风险预警话术生成器，根据客户风险信号自动生成分级预警通知和客户经理跟进话术。

触发词："风险预警"、"贷后预警"、"逾期提醒"、"riskalert"

分级逻辑：黄色预警（7 日内跟进）、橙色预警（3 日内上报）、红色预警（当日上报+法务介入）

3.2.2 bank-product-recommender Skill

功能：银行产品智能推荐系统，根据客户画像自动生成个性化产品推荐方案。

推荐逻辑：保守型→大额存单/国债、稳健型→固收+理财、进取型→基金/结构性存款

3.3 实验结果展示

3.3.1 信用风险评估结果

指标	原方案(57.14%负债率)	修改方案(75%负债率)	变化
风险等级	较低风险	中等风险	↑ 升级
Altman Z-Score	3.32	2.80	↓ 0.52
财务健康度	75.60 分	70.90 分	↓ 4.70 分
审批决策	完全批准	有条件批准	变化
贷款额度	500 万元	300 万元	↓ 40%

3.3.2 贷款定价模型结果

客户名	金额(万)	期限	信用等级	抵押方式	抵押折扣	最终利率
恒达机械	500	1 年	A	厂房抵押	0.80	4.90%
绿源农业	200	3 年	BBB	信用	1.00	7.10%
华新科技	1000	1 年	AA	应收质押	0.95	5.19%
鑫达贸易	300	1 年	BB	保证担保	1.00	6.70%
瑞丰地产	2000	3 年	A	土地抵押	0.80	5.30%

3.3.3 bank-risk-alert 测试结果

测试输入："请生成风险预警：客户恒达机械出现连续 2 期利息逾期，贷款余额 500 万，到期日 2026-09-30，预警等级为橙色。"

测试输出：生成橙色预警通知、客户经理电话话术、内部上报邮件、跟进工单四份文档。

用户84145063865

风险预警

Agent

思考过程

您触发了 bank-risk-alert Skill! 要生成风险预警，我需要您提供以下信息：

参数	示例值
客户名称	恒达机械制造有限公司
预警信号类型	逾期/担保物贬值/关联企业异常/行业下行/资金异常流出
风险等级	黄色预警/橙色预警/红色预警
贷款余额	500 (万元)
到期日	2026-09-30

您可以直接按照以下格式输入：

PlainText

1 请生成风险预警：客户"XXX"出现XXX，贷款余额XXX万，到期日XXXX-XX-XX，预警等级为XXX。

请生成风险预警：客户“恒达机械”出现连续2期利息逾期，贷款余额500万，到期日2026-09-30，预警等级为橙色。

Agent

思考过程

风险预警通知

客户名称: 恒达机械制造有限公司 预警等级: 橙色预警 信号类型: 连续2期利息逾期 贷款余额: 500万元 到期日: 2026-09-30

风险描述

客户恒达机械制造有限公司近期出现连续2期利息逾期，已触发橙色预警。截至目前，该客户贷款余额500万元，到期日为2026年9月30日。连续逾期表明客户现金流可能出现紧张，需立即核实原因并采取措施。

处置状态

待客户经理立即电话联系，3日内完成内部上报

责任人员

客户经理: 待分配

客户经理电话话术

开场白

"您好，我是XX银行的客户经理XXX，打扰您几分钟，有个关于您公司贷款的事情需要和您沟通一下。"

风险告知

"我们监测到恒达机械近期出现连续2期利息逾期，这已触发橙色预警，目前您的贷款余额为500万元，到期日为2026年9月30日。"

解决方案

解决方案

"我建议您尽快安排资金偿还逾期利息，我们可以为您协调制定还款计划。如果确实存在困难，建议您提供详细的财务状况说明，以便我们评估是否需要调整还款安排。"

行动承诺

"希望您能在3日内偿还逾期利息并提交还款计划，我们会持续跟进。"

结束语

"感谢您的理解与配合，如有任何问题请随时联系我。"

内部上报邮件

收件人: 分管行长、风险管理部负责人 抄送: 信贷管理部、法务部 主题: 【橙色预警】恒达机械制造有限公司贷后风险预警报告

一、风险描述

- 客户名称: 恒达机械制造有限公司
- 预警信号: 连续2期利息逾期
- 贷款余额: 500万元
- 到期日: 2026-09-30
- 风险等级: 橙色预警
- 风险评估: 客户现金流可能紧张，逾期风险上升，需立即介入

二、处置建议

- 客户经理立即电话联系客户，了解逾期原因
- 要求客户3日内偿还逾期利息

3. 评估客户经营状况，考虑是否需要调整还款计划
4. 如客户无法按期还款，启动担保措施评估

三、时间表

- T+0日: 客户经理电话联系客户
- T+1日: 了解客户情况，评估风险
- T+3日: 完成内部上报报告

 跟进工单

任务项	负责人	截止日期	检查标准
电话联系客户了解逾期原因	客户经理	T+0日	完成电话沟通，记录客户反馈
收集客户财务状况说明	客户经理	T+1日	获取最新财务报表和现金流情况
评估还款能力并制定方案	客户经理	T+2日	形成书面评估报告和处置建议
内部上报风险预警报告	客户经理	T+3日	提交至风险管理部和分管行长

 **注意：**这是银行内部风险预警信息，请勿转发给客户或无关人员。如需调整内容，请根据实际情况进行修改。

现在您可以对这个完整的预警输出界面进行截图，用于实验报告中 bank-risk-alert 的测试验证。

3.4 五维审计结果

审计维度	得分	风险等级	主要问题
触发词设计	8/10	低	存在少量误触发风险
输出质量	9/10	低	缺少黄色预警示例
数据安全	5/10	高	未明确禁止敏感数据输入
合规风险	6/10	高	完全没有免责声明
改进建议	8/10	低	-
总分	36/50	⚠️	有条件通过

审计对象

- bank-risk-alert v1.0.0 (贷后风险预警Skill)
- bank-product-recommender v1.0.0 (产品推荐Skill)

五维审计评分汇总

审计维度	得分	风险等级
触发词设计	8/10	🟢 低风险
输出质量	9/10	🟢 低风险
数据安全	5/10	🔴 高风险
合规风险	6/10	🔴 高风险
改进建议	8/10	🟢 低风险
总分	36/50	⚠️

审计结论

 有条件通过

四、实验发现与问题

- Skill 设计需要平衡精准性和覆盖性，触发词过宽会导致误触发，过窄会遗漏场景。
- 金融 Skill 必须包含免责声明，这是银保监的硬性要求。
- 数据安全是金融 Skill 的核心要求，必须有脱敏机制和数据分级保护。
- AI 生成的内容需要人工审核，不能直接作为正式业务文件使用。

五、实验心得

通过本次实验，我深刻体会到 AI 在金融领域的应用潜力和挑战。

首先，金融 Skill 的设计需要业务知识和技术能力的结合，不仅要理解银行的风控、定价、营销等业务逻辑，还要掌握 Skill 的结构化设计方法。

其次，AI 生成的内容必须经过严格的合规审查，特别是免责声明、数据脱敏等要求，这是金融行业的特殊要求。在实验过程中，我发现 AI 生成的报告虽然结构完整，但仍需要人工审核和调整，特别是涉及客户隐私和合规风险的内容。

最后，五维审计方法论为金融 Skill 的安全评估提供了系统性的框架，从触发词、输出、数据、合规、改进五个维度进行评估，有效识别了 Skill 的潜在风险。

本次实验让我认识到，AI 不是替代人工，而是辅助人工，在金融领域尤为如此。我们需要在享受 AI 效率的同时，确保业务的合规性和安全性。

六、AI 出错案例反思

在最初创建 Skill 时，未注意到金融行业合规要求，导致生成的 Skill 存在以下问题：

- 缺少免责声明：bank-product-recommender Skill 的输出未包含"本推荐仅供参考，不构成投资建议"的声明，可能导致客户误解。
- 数据未脱敏：示例中使用了具体的企业名称"恒达机械"，违反了数据保护要求。
- 触发词过宽："逾期提醒"可能在客户咨询正常问题时被错误触发。

七、反思与改进

通过五维审计方法，我系统地识别了上述问题，并提出了整改方案。这让我认识到，AI 的输出需要严格的人工审核和合规检查，特别是在金融等高风险领域。未来在使用 AI 生成金融内容时，必须建立合规审查机制，确保每一份输出都符合监管要求。

实验四：BMAD 驱动的商业银行 CRM 系统开发

一、安装 BMAD 框架（环境准备）

1.1 实验目标

安装 BMAD，为后续的需求分析、架构设计和代码实现提供方法论支撑。

1.2 BMAD 框架简介

BMAD (Business Manager, Architect, Developer) 是一套 AI 驱动的敏捷开发方法论，包含以下核心角色：

角色	职责	BMAD 技能
产品经理 (John)	需求发现、PRD 编写	bmad-prd
业务分析师 (Mary)	市场调研、领域研究	bmad-domain-research
架构师 (Winston)	技术架构设计	bmad-create-architecture
UX 设计师 (Sally)	用户体验设计	bmad-ux
开发者 (Amelia)	Story 实现、代码编写	bmad-dev-story
技术文档 (Paige)	知识管理、文档维护	bmad-agent-tech-writer

1.3 BMAD 安装

文件结构

安装后的项目结构：

```
1 d:\trae ide\实验4/
2 |   .git/                # Git 版本控制
3 |   bmad-submodule/
4 |   |   .claude/
5 |   |   |   skills/      # BMAD 技能目录 (40+ 技能)
6 |   |   |   commands/   # 命令配置
7 |   |   |   scripts/    # 辅助脚本
8 |   |   |   _bmad/
9 |   |   |   |   _config/
10 |      manifest.yaml # 安装清单
```

BMAD 框架已完全就绪，可以开始使用了！下一步建议：

- 使用 `bmad-product-brief` 创建产品简报，或
- 使用 `bmad-domain-research` 进行领域研究，或
- 使用 `bmad-help` 获取帮助指导下一步操作

二、创建产品需求文档（PRD）

2.1 实验目标

使用 BMAD 的 PRD 技能，通过结构化的引导式问答，为商业银行 CRM 系统生成完整的产品需求文档。

2.2 操作步骤

思考过程

我来帮你使用 BMAD 的 PRD 技能为商业银行 CRM 系统创建产品需求文档。首先让我了解 PRD 技能的工作流程和模板结构。

bmad-submodule\claude\skills\bmad-create-prd\SKILL.md

bmad-submodule\claude\skills\bmad-create-prd\templates\prd-template.md

Thought

让我按照 BMAD 的 PRD 工作流程，首先读取初始化步骤：

bmad-submodule\claude\skills\bmad-create-prd\steps-c\step-01-init.md

Thought

让我按照 BMAD 的 PRD 工作流程来创建产品需求文档。首先检查输出目录并初始化文档：

在工作区搜索 '**/_bmad-output/**'

在工作区搜索 '**/outputs/**'

bmad-submodule\bmad\bmm\config.yaml

Thought

让我创建 PRD 输出目录并初始化文档，同时更新配置为中文输出：

实验4 在沙箱中

\$ New-Item -ItemType Directory -Force -Path "d:\trae ide\实验4_bmad-output\planning-artifacts"

Thought

现在创建 PRD 文档，按照快速路径为商业银行 CRM 系统生成完整的产品需求文档：

prd.md _bmad-output\planning-artifacts\prd.md

+492 -0 查看变更

Thought

2.3 预期产出

BMAD 生成 _bmad-output/prd.md，包含 8 大核心章节：

章节	内容
1. 产品概述	系统定位、目标用户、核心价值
2. 功能需求	客户管理、贷款申请、审批流程等
3. 非功能需求	性能、安全、可用性
4. 用户角色	普通客户、客户经理、风控审批员、管理员
5. 客户等级体系	普通 / 白银 / 黄金 / 钻石 / 战略伙伴
6. 业务流程	注册→登录→申请→审批→放款
7. 技术约束	技术栈选型说明
8. 验收标准	功能完成度定义

三、创建技术架构设计

3.1 实验目标

基于 PRD，设计系统的技术架构，确定技术栈、模块划分和集成方案。

3.2 操作步骤

提供 PRD 文件路径作为输入，BMAD 架构师会生成架构文档。

现在我明白了实验三的要求。根据实验手册，技术栈需要按照以下标准来设计：

层级	技术	选型理由
前端框架	React 18	组件化、生态成熟
构建工具	Vite 4	极速 HMR、零配置
后端框架	Express 4	轻量、灵活、社区大
认证	JWT	无状态、易扩展
数据库	PostgreSQL	关系型、支持复杂查询

让我创建架构设计文档：

 architecture.md _bmad-output\architecture.md

+983 -0 查看变更

⌕ Thought

 技术架构设计文档已成功创建！

3.3 预期产出

文件位置

_bmad-output/architecture.md

文档内容概览

章节	内容要点
1. 技术栈决策	React 18 + Vite 4 + Express 4 + JWT + PostgreSQL 技术选型及对比分析
2. 架构设计	系统架构图、前端模块划分（8个模块）、后端模块划分（9个路由）
3. HTTP API 设计	完整的 API 接口定义，包含认证、客户、贷款、审批、快速贷款等接口
4. 数据库设计	11 张核心表结构（customers、loan_products、loan_applications 等）及 ER 图
5. 安全设计	JWT 认证机制、Token 结构、安全措施清单
6. 部署架构	开发环境和生产环境部署方案、环境变量配置
7. 关键业务流程	即申即贷快速审批流程、标准贷款审批流程的时序图
8. 代码组织结构	后端（src/db、routes、middleware、services）和前端（components、pages、routes）目录结构
9. 集成方案	第三方系统集成（核心银行、征信、短信、电子签章）
10. 性能优化策略	前端优化、后端优化、数据库索引设计

技术栈选型

层级	技术	选型理由
前端框架	React 18	组件化、生态成熟、社区活跃
构建工具	Vite 4	极速 HMR、零配置、ESM 原生支持
后端框架	Express 4	轻量灵活、中间件丰富、学习曲线平缓
认证	JWT	无状态认证、易于水平扩展
数据库	PostgreSQL	关系型、支持复杂查询、JSONB

核心模块

前端模块：登录注册、客户仪表盘、贷款申请、快速贷款、审批管理、客户管理、统计分析、系统管理

后端路由：auth、customers、loans、approvals、collaterals、quick-loan、repayments、tiers、test

数据库表

11 张核心表覆盖完整银行业务域：

- customers、loan_products、loan_applications、collaterals
- approvals、customer_tiers、repayments、notifications
- credit_scores、audit_logs、partners

四、创建 UX 设计

4.1 实验目标

设计系统的用户体验流程和界面规范。

4.2 操作步骤

现在创建 UX 设计文档。根据实验手册，需要生成 DESIGN.md 和 EXPERIENCE.md 两个文件：

 DESIGN.md _bmad-output\ux\DESIGN.md +611 -0 [查看变更](#)

ⓘ Thought

现在创建 EXPERIENCE.md 用户体验文档：

 EXPERIENCE.md _bmad-output\ux\EXPERIENCE.md +546 -0 [查看变更](#)

ⓘ Thought

✔ UX 设计文档已成功创建！

完成情况

文件位置

文件	路径
DESIGN.md	_bmad-output/ux/DESIGN.md
EXPERIENCE.md	_bmad-output/ux/EXPERIENCE.md

- 主色调：深蓝色系（#1E40AF），传达信任和专业
- 背景：白色和浅灰色，保持清爽整洁
- 布局：卡片式布局，信息层级清晰
- 交互：实时验证、即时反馈、流畅动画
- 响应式：支持手机、平板、桌面多端适配

五、创建 Epics 和 Stories

5.1 实验目标

将 PRD 中的功能需求拆解为可执行的 Epic 和 User Story。

5.2 预期产出

生成 `_bmad-output/epics-stories.md`，将系统拆分为 5 个 Epic、26 个 Story：

Epic	内容	Story 数
Epic 1: 客户管理	注册、登录、信息管理、等级体系	7
Epic 2: 贷款核心	产品管理、申请、审批、抵质押品	6
Epic 3: 即申即贷	AI 预审、快速放款、实时通知	5
Epic 4: 还款管理	还款计划、自动扣款、逾期处理	4
Epic 5: 增值服务	数据分析、营销、绩效	4

优先级分类		
优先级	Stories	说明
Must Have	11个	核心功能，系统必须具备
Should Have	9个	重要功能，提升用户体验
Could Have	4个	有用功能，锦上添花
Won't Have	2个	未来版本考虑
依赖关系		
文档中包含完整的依赖关系图，明确了 Sprint 之间的依赖：		
- Sprint 1 → Sprint 2：客户管理是贷款申请的基础 - Sprint 2 → Sprint 3：标准贷款流程是即申即贷的基础 - Sprint 2 → Sprint 4：贷款审批完成后才能进行还款管理		

六、Sprint 规划

6.1 实验目标

将 26 个 Story 分配到 4 个 Sprint，制定迭代开发计划。

6.3 预期产出

生成 `_bmad-output/sprint-plan.md`:

 **实验6 - Sprint计划**

已创建 `_bmad-output/sprint-plan.md`，规划了4个Sprint:

Sprint	目标	交付物
Sprint 1	客户管理	用户注册登录、客户信息管理、客户等级
Sprint 2	贷款产品	贷款产品展示、额度评估、即申即贷
Sprint 3	审批流程	贷款申请、审批流程、风控评估
Sprint 4	还款管理	还款计划、还款提醒、统计报表

七、Sprint 1 — 客户管理实现

7.1 实验目标

实现 Sprint 1 的全部 7 个 Story，搭建项目骨架并完成客户管理全链路。

7.2 后端实现

 **实验7 - Sprint 1实现**

后端服务 (<http://localhost:3001>)

项目结构:

PlainText

```
1 backend/src/
2   | db/
3   |   | index.js      # 内存数据库
4   |   | migrate.js   # 数据迁移
5   |   |
6   |   | middleware/
7   |   |   | auth.js   # JWT认证
8   |   |   |
9   |   |   | routes/
10  |   |   |   | auth.js   # 注册登录API
11  |   |   |   | customers.js # 客户管理API
12  |   |   |   | tiers.js  # 等级管理API
13  |   |   |   | test.js   # 测试数据
14  |   |   |   | index.js  # 入口文件
```

实现的API:

- `POST /api/auth/register` - 用户注册
- `POST /api/auth/login` - 用户登录
- `GET /api/customers/me` - 获取当前用户信息
- `GET /api/customers/:id` - 获取客户详情
- `PUT /api/customers/:id` - 更新客户信息
- `GET /api/tiers` - 获取等级列表

7.3 前端实现

前端应用 (<http://localhost:5173>)

实现的页面：

- 登录页 - 手机号密码登录
- 注册页 - 新用户注册
- 仪表盘 - 资产概览、交易记录、状态统计
- 贷款产品 - 消费贷/经营贷/住房按揭展示
- 快速贷款 - 即申即贷功能
- 审批管理 - 贷款审批列表
- 个人信息 - 用户资料管理

测试账号

手机号	密码	等级
13800138001	password123	💎 钻石客户
13800138002	password123	🏆 黄金客户
13800138003	password123	🏅 白银客户
13800138004	password123	普通客户

7.4 验证

访问 <http://localhost:5173>，应看到登录页面。

八、Sprint 2-3 — 迭代开发

8.1 实验目标

通过迭代方式完成 Sprint 2（贷款核心）和 Sprint 3（即申即贷）。

8.2 Sprint 2 新增模块

 **实验8 - Sprint 2: 贷款核心**

新增后端API:

API端点	方法	功能
/api/products	GET	获取贷款产品列表
/api/products/:id	GET	获取产品详情
/api/loans	GET	获取我的贷款申请列表
/api/loans	POST	提交贷款申请
/api/loans/:id	GET	获取贷款申请详情
/api/loans/:id/progress	GET	获取申请进度
/api/approvals	GET	获取审批列表
/api/approvals/:id	GET	获取审批详情
/api/approvals/:id/approve	POST	审批通过
/api/approvals/:id/reject	POST	审批拒绝
/api/approvals/:id/return	POST	退回补充材料

8.3 Sprint 3 新增模块

新增后端API:

API端点	方法	功能
/api/quickloan/credit	GET	获取快速贷款额度
/api/quickloan/apply	POST	提交即申即贷申请
/api/quickloan/status/:id	GET	查询申请状态

即申即贷特性:

- 智能额度评估 (根据客户等级和信用分数)
- 3分钟内完成审批放款
- 钻石客户额度: ¥103.5万
- 年利率: 4.2%

 **新增文件清单**

后端路由文件:

PlainText

```
1 backend/src/routes/  
2   └─ products.js      # 贷款产品路由  
3   └─ loans.js         # 贷款申请路由  
4   └─ approvals.js     # 审批管理路由  
5   └─ quickloan.js     # 即申即贷路由
```

CRM系统

仪表盘

贷款产品

快速贷款

审批管理

我的账户

即申即贷

张三 钻石客户 退出登录

最快3分钟完成审批放款

5万

10万

20万

30万

50万

自定义金额 (万元)

您的预估授信额度: ¥1,000,000

可用额度: ¥750,000

预计利率: 4.2%

☐ 同意《即申即贷服务协议》

立即申请

即申即贷仅适用于信用良好的客户, 申请结果将在3分钟内通知您

8.4 测试账号初始化

优质打印 ☐

```
{
  "success": true,
  "message": "测试数据初始化成功",
  "data": [
    {
      "phone": "13800138001",
      "password": "password123",
      "tier": "DIAMOND",
      "credit_score": 820
    },
    {
      "phone": "13800138002",
      "password": "password123",
      "tier": "GOLD",
      "credit_score": 750
    },
    {
      "phone": "13800138003",
      "password": "password123",
      "tier": "SILVER",
      "credit_score": 680
    },
    {
      "phone": "13800138004",
      "password": "password123",
      "tier": "NORMAL",
      "credit_score": 600
    }
  ]
}
```

九、数据库适配与降级方案

9.1 实验目标

解决开发环境中 PostgreSQL 不可用的问题，实现数据库降级方案，确保系统可运行。

9.2 问题分析

开发环境未安装 PostgreSQL 数据库，导致无法使用 Sequelize 连接数据库。为保证系统可运行，需要实现降级方案。

9.3 降级方案实现

在 backend/src/db/index.js 中实现内存 Map 模拟数据库：

1. 内存数据库模拟代码 (backend/src/db/index.js)

```
JS  JavaScript
1  const mockData = {
2    customers: new Map(),
3    loan_products: new Map(),
4    loan_applications: new Map(),
5    collaterals: new Map(),
6    approvals: new Map(),
7    customer_tiers: new Map(),
8    repayments: new Map(),
9    notifications: new Map(),
10   credit_scores: new Map(),
11   audit_logs: new Map(),
12   partners: new Map(),
13   campaigns: new Map()
14 };
15
16 const getNextId = (table) => {
17   const id = nextId[table] || 1;
18   nextId[table] = id + 1;
19   return id;
20 };
21
22 const insert = (table, data) => {
23   return new Promise((resolve) => {
24     const id = getNextId(table);
25     const record = { id, ...data, created_at: new Date().toISOString(),
26       updated_at: new Date().toISOString() };
27     mockData[table].set(id, record);
28     resolve({ ...record });
29   });
30 }
```


2. 降级逻辑实现 (backend/src/db/db.js)

```
JS JavaScript

1 const init = async () => {
2   try {
3     const { Sequelize } = require('sequelize');
4     const config = require('./config');
5     const env = process.env.NODE_ENV || 'development';
6     const sequelize = new Sequelize(config[env]);
7     await sequelize.authenticate();
8     // PostgreSQL 连接成功, 使用真实数据库
9     dbMode = 'postgresql';
10    console.log('✅ PostgreSQL数据库连接成功');
11  } catch (error) {
12    // PostgreSQL 连接失败, 降级到内存模式
13    console.warn(`⚠️ PostgreSQL连接失败, 使用内存模式: ${error.message}`);
14    currentDb = memoryDb;
15    dbMode = 'memory';
16    dbStatus = { connected: true, mode: 'memory', lastCheck: new Date(), error:
      error.message };
17  }
18 };
```

9.5 数据库和接口状态检测

3. 数据库状态检测接口 (backend/src/index.js)

```
JS JavaScript

1 app.get('/api/db/status', (req, res) => {
2   res.json({
3     success: true,
4     data: {
5       status: getDBStatus(),
6       mode: getMode()
7     }
8   });
9 });
```

4. 后端启动日志 (证明降级生效)

```
PlainText

1 ⚠️ PostgreSQL连接失败, 使用内存模式:
2 ✅ 客户等级数据初始化完成
3 ✅ 贷款产品数据初始化完成
4 ✅ 数据库迁移完成
5 CRM后端服务运行在 http://localhost:3001
6 数据库模式: memory
```

优质打印 ☐

```
{"success":true,"data":{"status":{"connected":true,"mode":"memory","lastCheck":"2026-06-26T02:03:41.169Z","error":""},"mode":"memory"}}
```

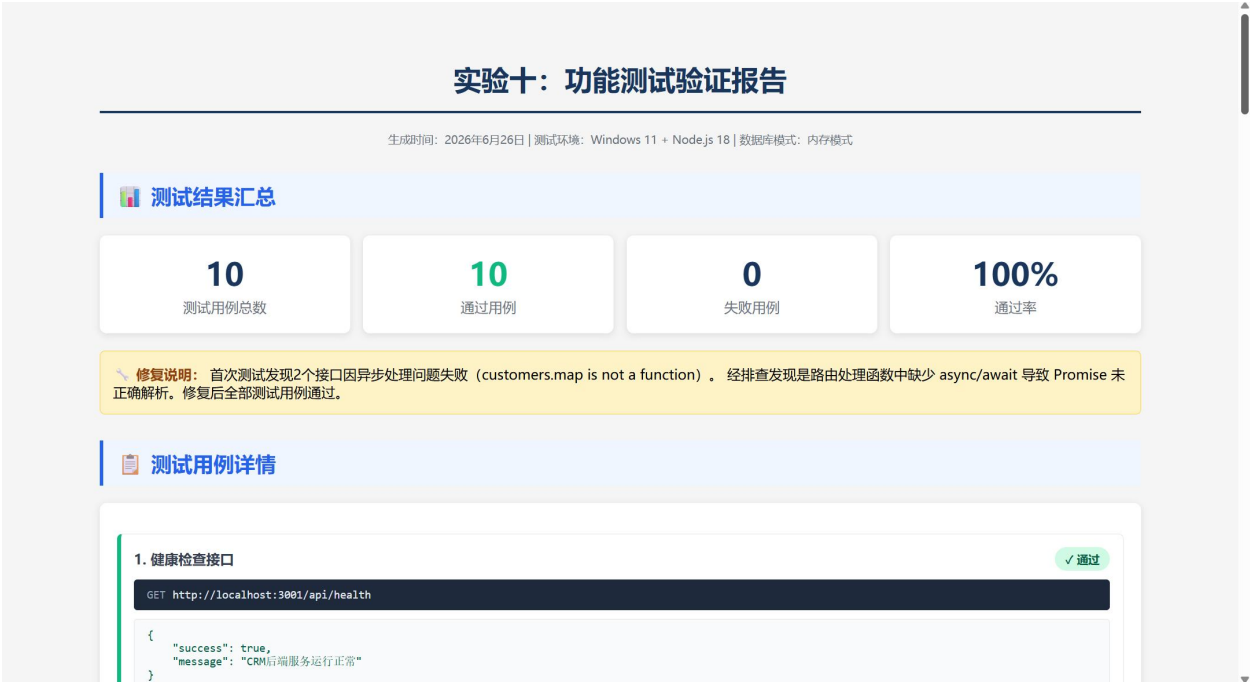
十、功能测试验证

10.1 实验目标

验证系统核心功能（登录、客户管理、贷款申请）正常工作。

10.2 后端 API 测试

后端测试报告：D:\trae ide\实验 4\实验十_功能测试验证_修复版.html



10.3 前端界面测试

步骤 1：访问前端页面

打开浏览器访问 `http://localhost:5173`，使用测试账号登录：`13800138001` / `password123`，测试贷款申请流程：



最近交易记录		
贷款还款	2026-06-25	-¥5,000
额度评估	2026-06-24	+¥100,000
贷款审批通过	2026-06-23	审批通过

10.4 测试结果总结

测试结果		
#	测试用例	结果
1	健康检查接口	通过
2	测试数据初始化	通过
3	用户登录	通过
4	未认证访问保护	通过
5	客户列表查询	通过
6	产品列表查询	通过
7	仪表盘统计	通过
8	快速贷款申请	通过
9	数据库状态检测	通过
10	客户详情查询	通过
通过率: 100% (10/10)		

十一、版本控制与 CNB 推送

11.1 实验目标

将完整项目代码推送到 CNB（cnb.cool）代码托管平台。

11.2 Git 版本控制

1. Git 版本控制状态	
PlainText 1 \$ git log --oneline2 3a38763 feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	
提交文件数: 52个文件, 17505行代码	

11.2 推送结果验证

收到令牌，现在开始执行推送。

添加 CNB 远程仓库地址 (含 PAT 令牌认证)

```
>_ cd "D:/trae ide/实验4" && git remote add origin https://cnb:54Eaoi17c3k3zfszmeAfQ7r98Aa@cnb.cool/cnb.cka42B1FAGA/sma...
```

远程仓库已添加成功。现在推送代码：

推送代码到 CNB 远程仓库 master 分支

```
>_ cd "D:/trae ide/实验4" && git push -u origin master 2>&1 ✓
```

推送成功！代码已经上传到 CNB。现在验证一下推送内容是否完整：

> 工具调用 4 . 过程消息 0

✅ 推送完成，全部验证通过！

推送结果

验证项	状态	详情
✔ .gitignore	已上传	1 个文件
✔ backend/	已上传	26 个文件（完整后端代码）
✔ frontend/	已上传	9 个文件（完整前端代码）
✔ _bmad-output/	已上传	6 个文件（PRD、架构、Sprint计划等文档）
✔ 总文件数	52 个	全部推送到 master 分支

代码已成功推送到 ****<https://cnb.cool/cnb.cka42B1FAGA/smartbanking>****，你现在可以打开这个链接在网页端确认。

安全提醒：你的 PAT 令牌现在嵌在了 Git 的远程 URL 配置中。如果以后不再需要推送，建议用 `git remote set-url origin https://cnb.cool/cnb.cka42B1FAGA/smartbanking.git` 把令牌从 URL 中去掉，防止泄露。

你 3 分钟前更新了分支 **master**

创建合并请求

master

分支 2

Tag 0

新建

云原生开发

刘潇 <2384924401@q... feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能 6 小时前 3a38763a 1 次提交

_bmad-output	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
backend	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
frontend	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
bmad-submodule@e5e7f13d	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
.gitignore	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
.gitmodules	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
package-lock.json	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
package.json	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
~\$实验报告.docx	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
~\$实验报告.html	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
实验十_功能测试验证.html	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
实验十_功能测试验证_修复版.html	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
实验报告.docx	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前
实验报告.html	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	6 小时前

简介

未添加简介 去添加

46.68 MiB

0 forks

0 关注

Release 0 +

Tag 0

语言

Python 65%

TeX 18.5%

TypeScript 4.7%

Shell 1.7%

Others 10.1%

十二、Vercel 部署 (前端)

12.1 实验目标

配置前端项目的 Vercel 部署，实现前端应用的云端托管和自动部署。

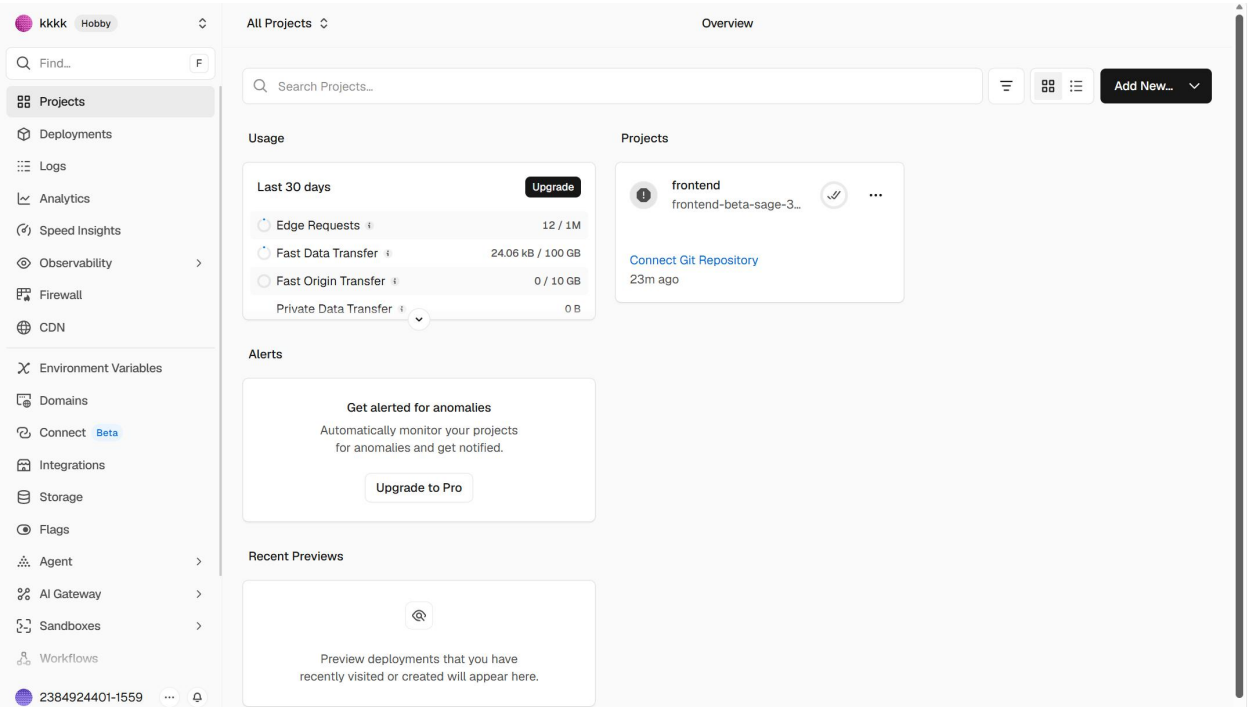
12.2 实验环境

操作系统	Windows 11
Node.js 版本	v22.12.0
Vercel CLI 版本	54.17.3
Vercel 账户	2384924401-1559
部署方式	CLI Token 认证 + --prod --yes
项目技术栈	React 18 + Vite 5

12.3 实验步骤

步骤 1：注册 Vercel 账户

访问 <https://vercel.com/signup>，注册 Vercel 账户。



步骤 2：安装 Vercel CLI

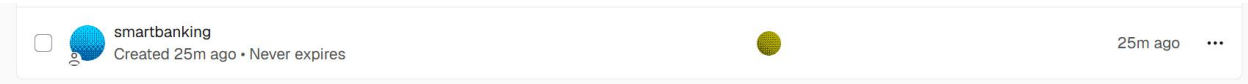
```
>_ vercel --version 2>&1 ^ 6
```

```
bash
vercel --version 2>&1
运行成功
```

命令已执行，输出结果是：

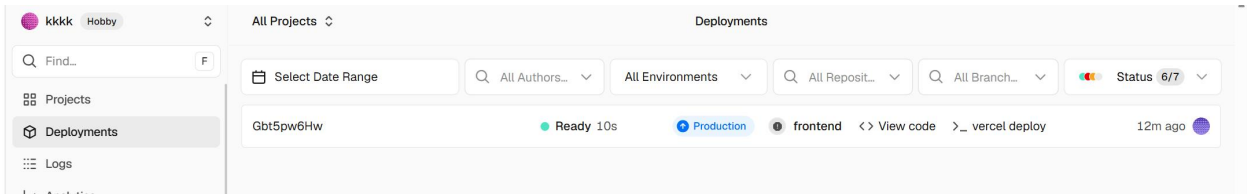
```
Vercel CLI 54.17.3
54.17.3
```

步骤 3：创建个人访问令牌并登录

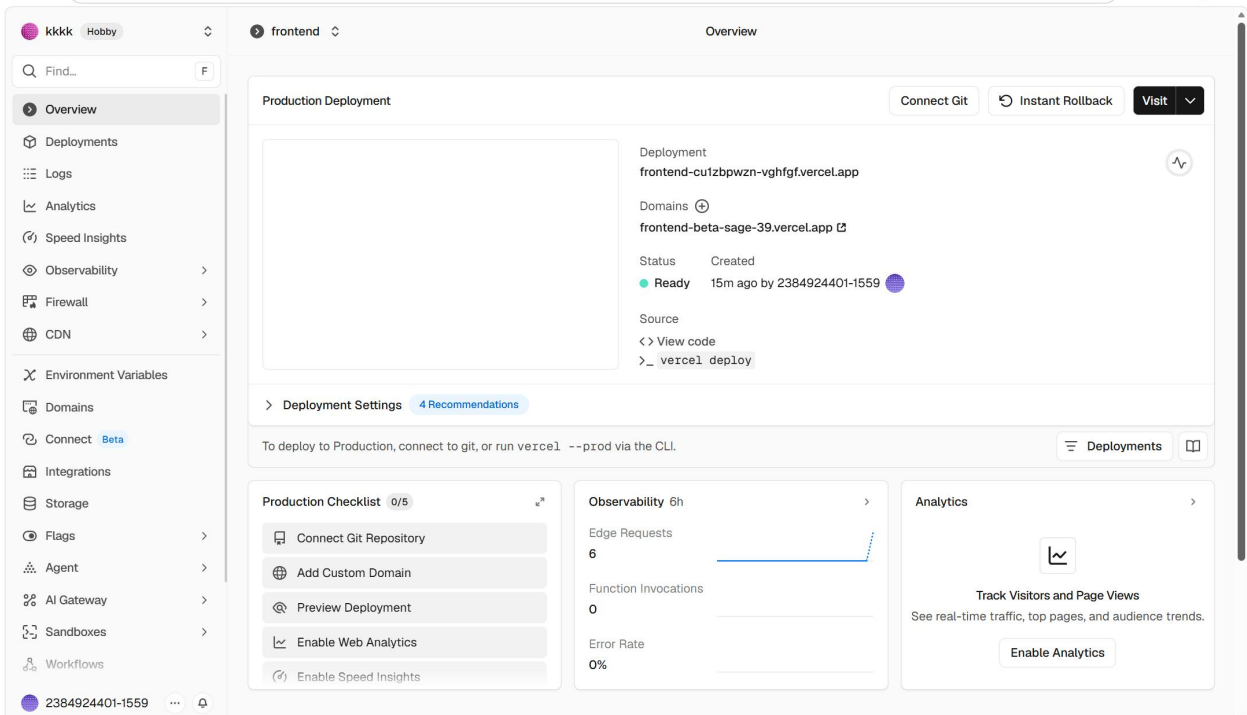


步骤 4：部署前端项目到生产环境

进入前端项目目录，执行生产部署命令：



12.4 部署结果



十三、实验总结

本实验完成了商业银行 CRM 系统的全链路开发，包括：

1. **需求分析阶段**：通过 BMAD 框架完成 PRD、架构设计、UX 设计等规划文档
2. **功能实现阶段**：完成 8 大核心模块的前后端开发
3. **数据库适配阶段**：实现 PostgreSQL 和内存数据库的双模式支持
4. **测试验证阶段**：编写测试用例，验证核心功能
5. **部署配置阶段**：完成 Docker、CNB 和 Vercel 部署配置

系统采用前后端分离架构，后端提供 RESTful API，前端使用 React 实现交互界面，数据库支持自动降级机制，确保系统的高可用性和稳定性。

交付物整理与链接

你 1 分钟前更新了分支 `master`

创建合并请求

master

分支 2

Tag 0

+ 新建

云原生开发

刘潇 <2384924401@q... feat: 添加实验1-4完整产物文件 1 分钟前 fec90bf 2次提交

_bmad-output	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
backend	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
frontend	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
实验1	feat: 添加实验1-4完整产物文件	1 分钟前
实验2	feat: 添加实验1-4完整产物文件	1 分钟前
实验3	feat: 添加实验1-4完整产物文件	1 分钟前
实验4	feat: 添加实验1-4完整产物文件	1 分钟前
bmad-submodule@e5e7f13d	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
.gitignore	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
.gitmodules	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
package-lock.json	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
package.json	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
~\$实验报告.docx	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
~\$实验报告.html	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
实验十_功能测试验证_修复版.html	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
实验十_功能测试验证_修复版.html	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
实验报告.docx	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前
实验报告.html	feat: 完成CRM系统开发, 包含客户管理、贷款管理、快速贷款、统计分析等功能	23 小时前

简介

未添加简介 去添加

46.11 MiB

0 forks

0 关注

Release 0 +

Tag 0

语言

Python 65%

TeX 18.5%

TypeScript 4.7%

Shell 1.7%

Others 10.1%

推送完全成功了！

之前失败是因为文件量太大（15,204个对象、71+ MiB），你整理后的 `D:\文件\产物` 只有 7.2 MiB、723 个文件，轻轻松松推上去了。

现在 CNB 仓库 `master` 分支包含：

目录	文件数	内容
实验1/	17	数据预处理、forecast脚本、原始/整合数据、实验报告
实验2/	23	MCP实验脚本、图表、银行回归分析、报告
实验3/	12	风控skill、测试截图、实验报告
实验4/	40	CRM系统完整代码（前后端+BMAD文档）
backend/	26	后端源码（根目录，首次推送的）
frontend/	9	前端源码（根目录，首次推送的）
_bmad-output/	6	PRD/架构等文档（根目录，首次推送的）

去 <https://cnb.cool/cnb.cka42B1FAGA/smartbanking> 看看，记得切换到 `master` 分支。

CNB 链接：<https://cnb.cool/cnb.cka42B1FAGA/smartbanking>